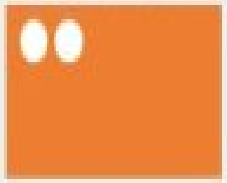
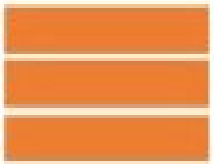
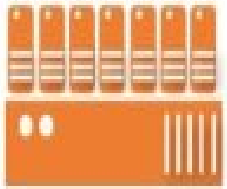
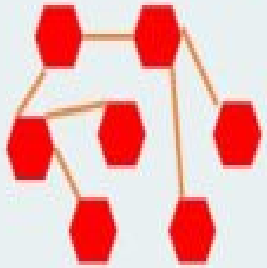
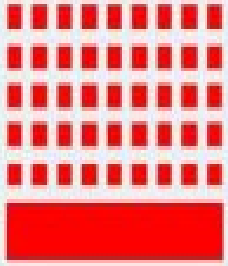
Microservices and containers

- **Monolithic and Microservices**

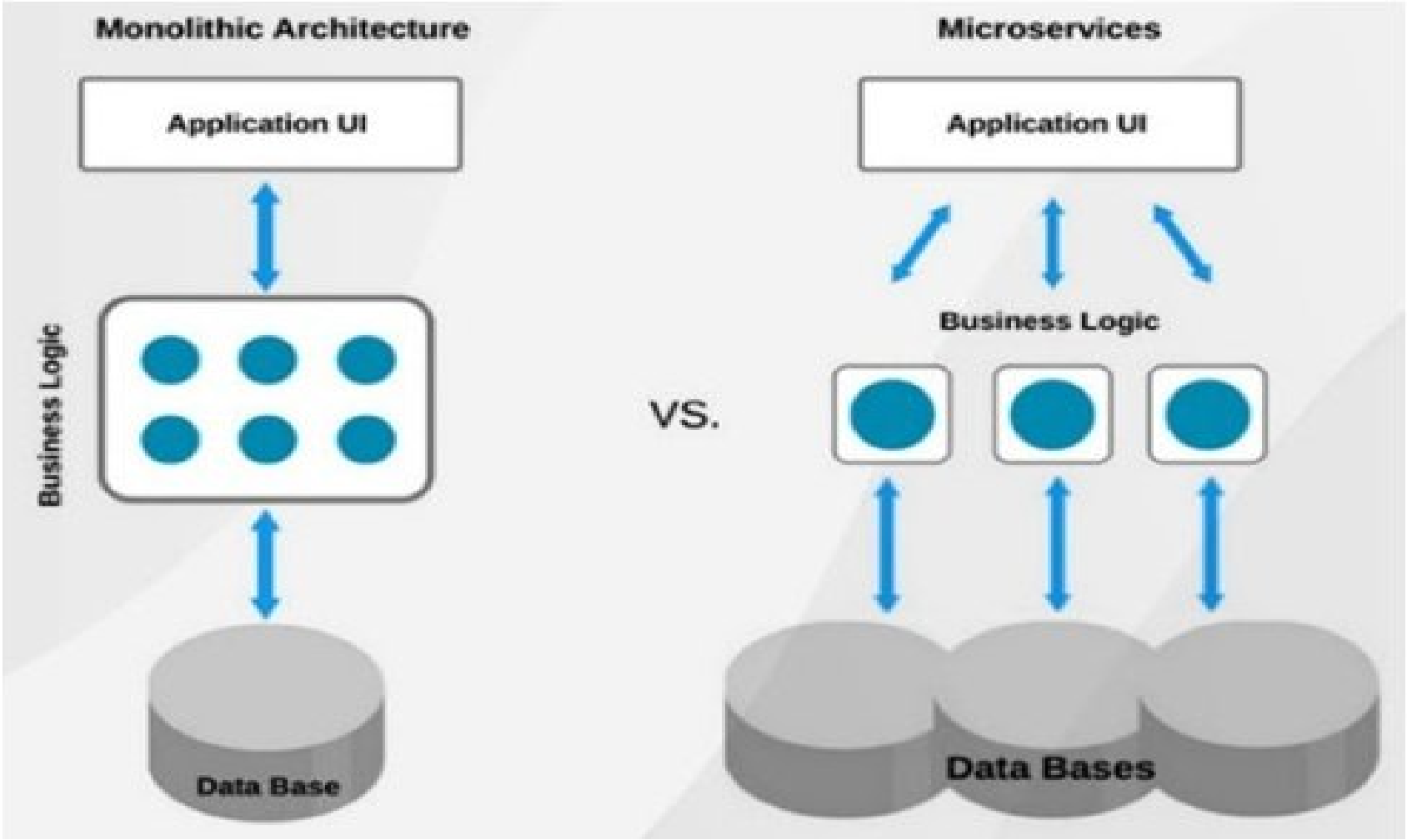
- **There has been a lot of change in the development & deployment since the ’80s till now. We can trace the evolution of how organizations have developed, deployed, and managed applications over three distinct phases:**

	Development Process	Application Architecture	Deployment and Packaging	Application Infrastructure
~ 1980 ~ 1990	Waterfall 	Monolithic 	Physical Server 	Datacenter 
~ 2000	Agile 	N-Tier 	Virtual Servers 	Hosted 
~ 2010 Now	DevOps 	Microservices 	Containers 	Cloud 

- 80's – mid 90's: Data Centers with physical servers, running monolithic applications, developed using traditional waterfall methodologies.

- Late 90s–00s: Datacenters and hosting providers running Unix/Linux servers, the emergence of virtualization (VMWare, KVM), running n-tier applications, developed using agile methodologies.

- ~ 2010 – Now: Cloud increasingly replacing data centers and hosting, decomposition of applications into microservices, running on a container infrastructure, managed and orchestrated by Kubernetes, developers, and operations collaborating using DevOps methodologies.


Monolithic Architecture
Application UI
Business Logic
Data Base
VS.
Microservices
Application UI
Business Logic
Data Bases

Monolithic Application

- A monolithic application is constructed as one unit which means it's composed all in one piece. The Monolithic application describes a one-tiered software application within which different components combined into one program from a single platform. The three components are the user interface, the data access layer, and the data store.
- The user interface is the entry point of the application and is also the point of the program that a user will interact with.
- Data access layer is the layer of the program which will wrap the data stored.
- The data store is the most fundamental part of the system and is liable for storing data and retrieving it.

Example of a Monolithic Application

An e-commerce website like early versions of Amazon or Flipkart (before adopting microservices).

- The entire system — user interface, business logic, and database access — was built as one single codebase and deployed together.
- If developers needed to update the "Add to Cart" feature, they had to rebuild and redeploy the whole application.

1. User Interface (UI) Example

- Example: The web pages or app screens that customers use to browse products, view prices, and click “Buy Now.”
- This part displays information to users and collects their input.
- Technologies (example): HTML, CSS, JavaScript (front-end built into the same application).

2. Data Access Layer Example

- Example: A set of functions or modules in the application that handle communication between the UI and the database.
  - For instance, a ProductDAO class (Data Access Object) retrieves product details from the database.
- Technologies (example): Java JDBC, Python's SQLAlchemy, or .NET Entity Framework within the same codebase.

3. Data Store Example

- **Example: A relational database like MySQL or PostgreSQL that stores all data — user accounts, product details, and orders.**
- **The application connects directly to this single database to read and write data.**

Advantages of Monolithic:

- **Simple to deploy** – In monolithic applications, we don't need to handle many deployments just one file or directory.

- **Easier debugging and testing** – monolithic applications are much easier to debug and test. Since a monolithic app is a single indivisible unit, we can run end-to-end testing much faster.

- **Performance:** Components in a monolith typically share memory which is Quicker than service-to-service communications.

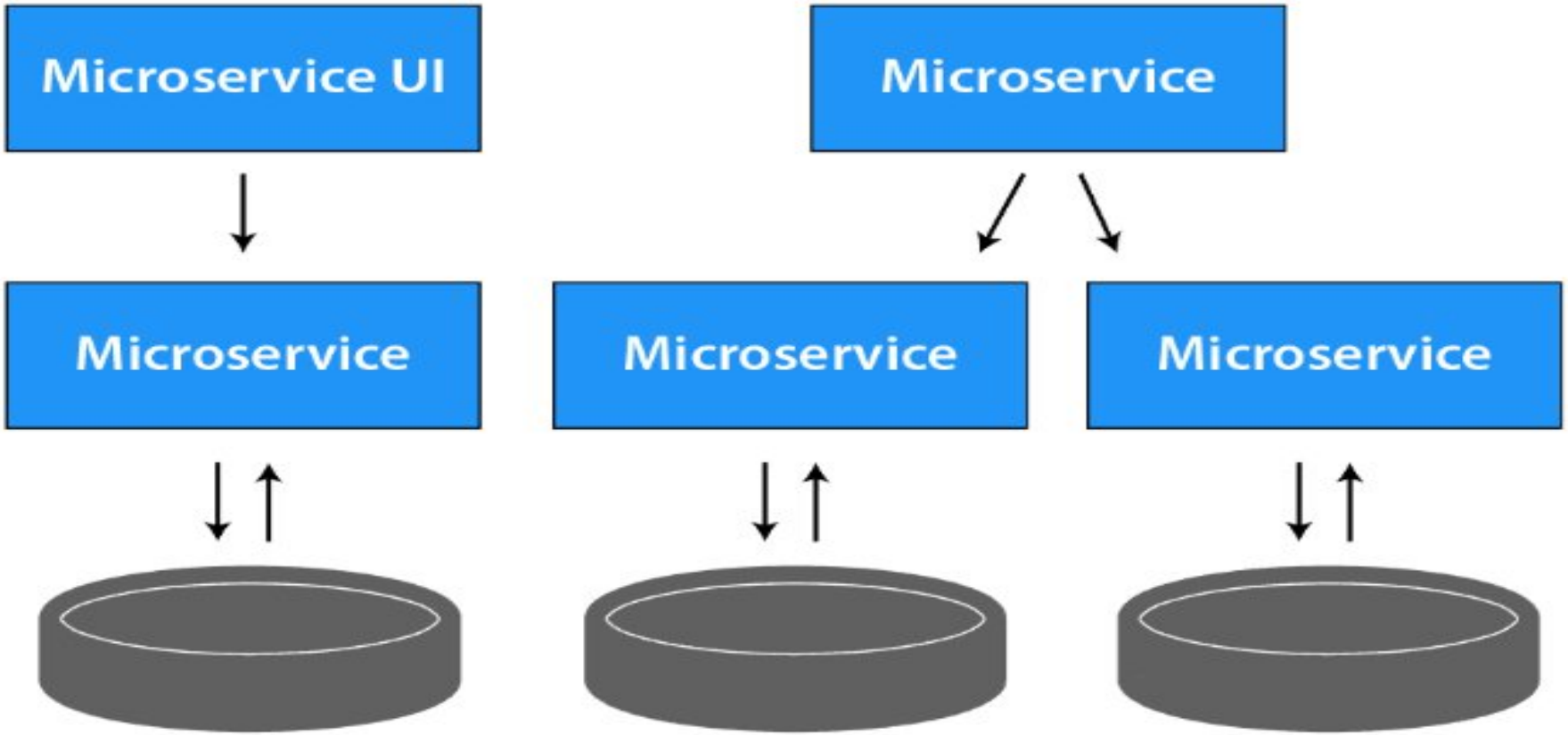
Disadvantages of Monolithic:

- **New technology barriers – It's extremely problematic to use new technology in a monolithic application because then the whole application must be rewritten.**

- **Scalability- You can't scale components independently, only the whole application.**

- **Size – It becomes overlarge in size with time and becomes difficult to manage.**

- **Difficult to understand – For any new developer joining the project, it's very difficult to grasp the logic of an oversized monolithic application whether or not his responsibility is related to one functionality.**



Micro-services Architecture

a) The idea of microservices is to separate our application into smaller sets and interconnected services rather than building one monolithic application.

b) Each module supports a selected business goal and uses a simple and well-defined interface to communicate with other sets of services.

c) Each microservice has its own database. Having a database per service is crucial if you wish to learn from microservices because it ensures loose coupling.

Example of a Microservices Application

- **Example: *Modern Amazon or Netflix***
  - **Instead of one large application, they're made up of many small services, such as:**
    - **User Management Service**
    - **Product Catalog Service**
    - **Payment Service**
    - **Order Tracking Service**
    - **Recommendation Service**
  - **Each service runs independently but communicates with others using APIs.**

Smaller and Interconnected Services

Example:

- In an online shopping platform, separate services are built for:
  - User Service → handles login, signup, profile.
  - Order Service → manages order placement and history.
  - Payment Service → processes payments and refunds.
- These services communicate using REST APIs or message queues.

Each Module Supports a Business Goal

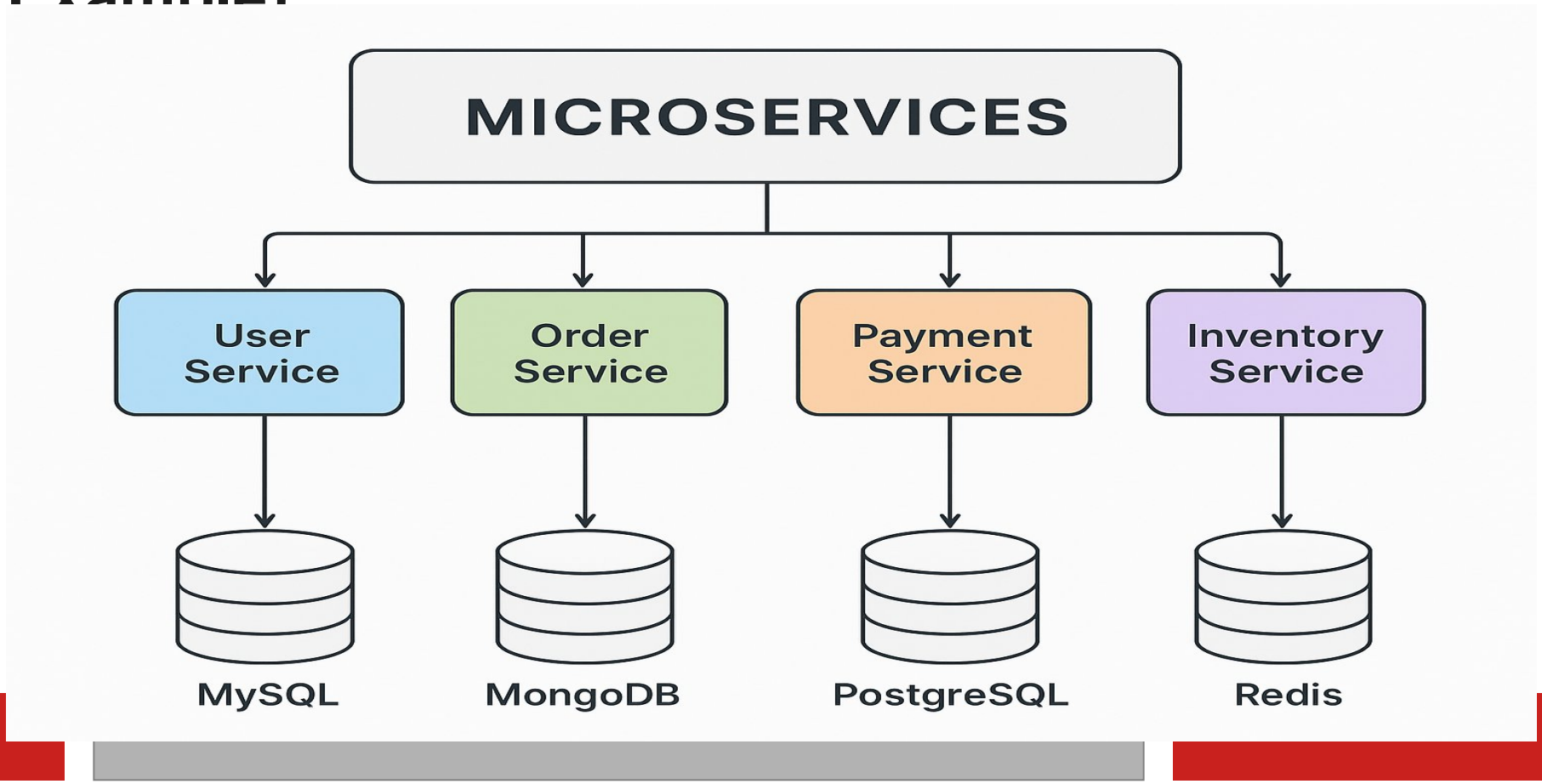
- **Example:**
  - **The Payment Service has one clear goal: handle all payment transactions.**
  - **The Inventory Service focuses only on updating and managing stock levels.**
  - **Each module (microservice) does one thing well and can be developed, deployed, and scaled independently.**
- **Communication Example:**
  - **When a user places an order, the Order Service calls the Payment Service via an API like:**
  - **POST /payment/process**
  - **The Payment Service confirms the transaction and notifies the Order Service to proceed.**

Each Microservice Has Its Own Database

- **Example:**
  - **User Service Database (MySQL): Stores usernames, emails, passwords.**
  - **Order Service Database (MongoDB): Stores order details, product IDs, delivery status.**
  - **Payment Service Database (PostgreSQL): Stores payment transactions and invoices.**
- **This ensures loose coupling — one service can change its database or schema without affecting others.**

Other Example :

Online Food Delivery App (Microservices Example):



Advantages of Microservices:

Advantage	Explanation	Example	Benefit
1. Scalability	Each microservice can be scaled independently based on demand.	During festive sales, only the **Order Service** is scaled up while others remain constant.	Efficient resource use and better performance.
2. Independent Deployment	Teams can deploy or update one microservice without redeploying the whole system.	**Payment Service** is updated for new payment gateway without affecting **User** or **Order** services.	Faster updates and reduced downtime.
3. Technology Flexibility	Each service can use a different tech stack suitable for its task.	**User Service** in Python, **Order Service** in Java, **Payment Service** in Node.js.	Freedom to choose best-fit tools and technologies.

4. Fault Isolation	A failure in one service does not crash the entire application.	If Recommendation Service fails, other services like Order and Payment still work.	Increased reliability and system uptime.
5. Faster Development & Deployment	Smaller, independent teams can develop, test, and deploy services in parallel.	One team works on Inventory, another on Payment, both releasing updates independently.	Accelerates innovation and delivery speed.
6. Easier Maintenance & Debugging	Each service is smaller and easier to test or fix.	Bugs in User Service can be fixed quickly without touching other modules.	Simplifies debugging and reduces risk.

DISADVANTAGES OF MICROSERVICES ARCHITECTURE

1. Complexity

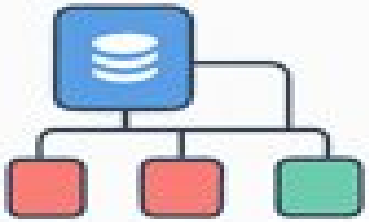
Managing many services and interactions can be complicated

2. Increased Latency

Multiple inter-service calls can add latency to the system

3. Data Consistency

Ensuring data consistency across services can be challenging

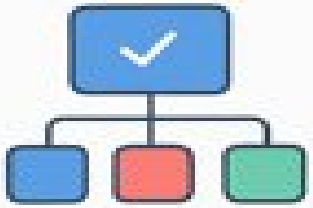


Data Management Complexity

Since each microservice has its own database, maintaining data consistency is hard.

Ex: Updating order status in Order Service must also reflect in Payment Service database

Ex: Risk of inconsistent data of complex transactions

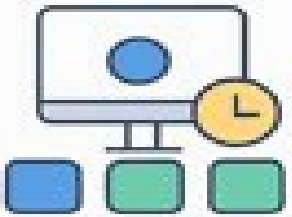


Testing Challenges

Testing across many services is harder than testing a single monolithic application

Ex: End-to-end testing reguires running User. Order, and Payment services together

Ex: More complicated integration and regression testing



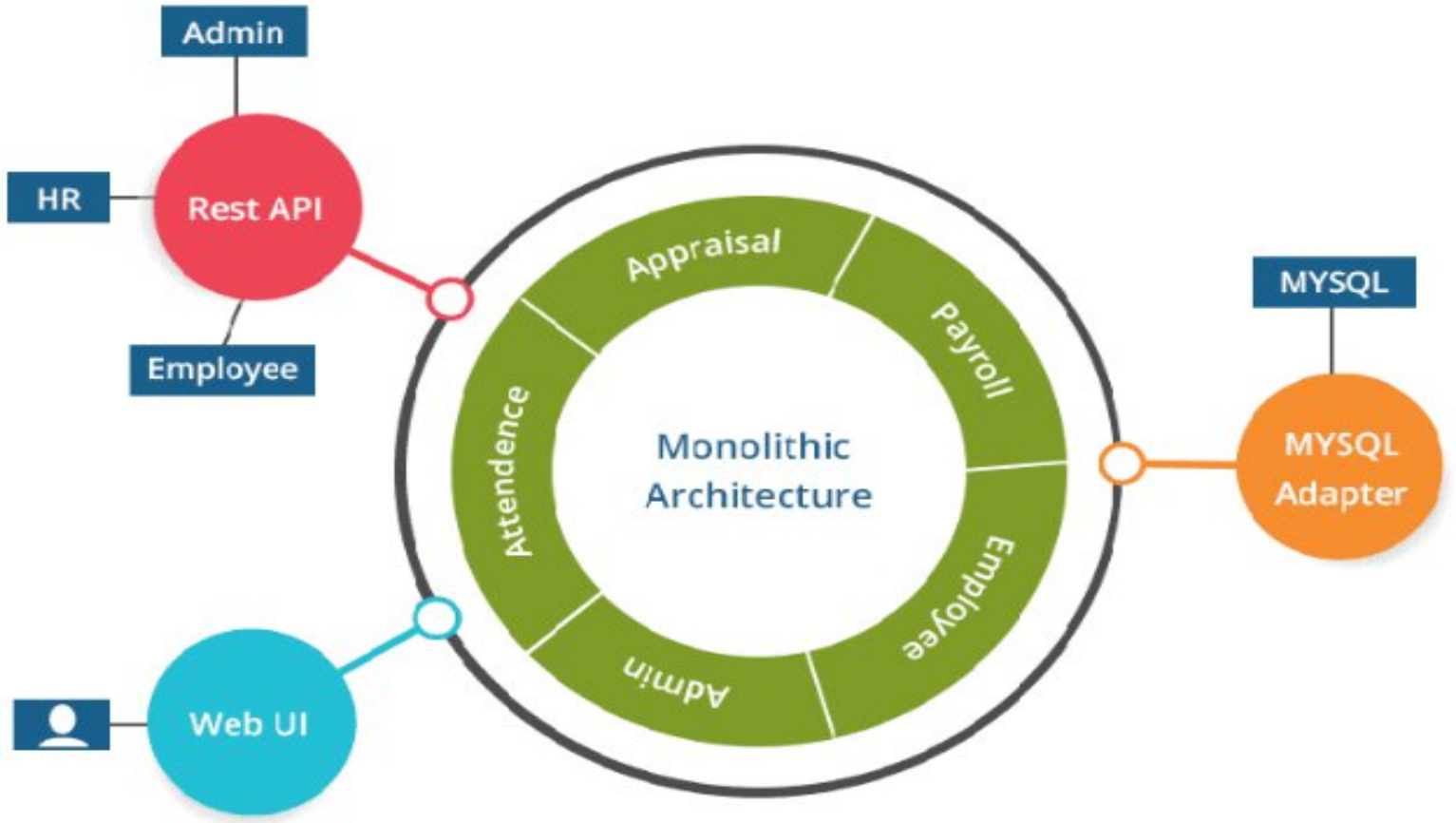
Increased Resource Usage

Each service runs in its own container or process, consuming extra memory and CPU

Ex: 20 microservices rhning in Docker use more system resources than one monolithic app

Ex: Can increase infrastructure cost

Why MicroServices? ^



- The good thing about using microservices is that development teams are ready to rapidly build new components of apps to fulfill changing business needs.

- In microservices, every application resides in separate containers along with the environment it needs to run.

- It allows it to maximize deployment velocity and application reliability.

- There's no risk of interfering with the other applications. It also allows optimizing the sources micro services multiply team works on independent services.

- In Microservices, we run all the applications on Containers (Docker), so the boot-up time and memory consumption of applications decrease, thus the performance of the application increases.

- In organizations, not one or two but multiple numbers of Containers (Docker) are running.

- So, to manage them, we use Kubernetes. Kubernetes comes in handy because it provides various facilities like self-healing, scalability, auto-scaling, and declarative state.

What's the Diff: Monolithic vs Microservices

Monolithic	Microservice
It is built as one large system	It is built as a small independent module
Not easy to scale based on demand	Easy to scale based on demand.
It has a shared database	Each project and module have their own database
Large code base makes IDE slow	Each project is independent and small in size
Continues deployment becomes difficult	Continues deployment is possible here
It is extremely difficult to change technology or language or framework because everything is tightly coupled and depends on each other.	Easy to change technology or framework because every module and project is independent.

Point of Discussion

1. If an online shopping platform on AWS faces high demand for its payment service, how would scaling differ for monolithic vs. microservices architecture?

2. Can you explain what happens during a big sale event—how might microservices help Flipkart manage traffic surges differently than a monolithic system?

Concept of Containers

- Containers are Operating System-level virtualization technologies that allow multiple isolated applications to run on a single host or instance.

- They provide agile, cost efficient, and automated application deployment in which processes are packaged with all the necessary dependencies inside their own separate instances for easy access and portability.

- Containers also offer greater levels of isolation between resources and improve resource utilization compared to traditional virtual machines by using fewer hardware resources.

- Containers are a lightweight alternative to VMs for providing **isolated** operating environments for your workloads.
- They use a different method of **abstracting** resources.
- Instead of using a hypervisor, containers share the **kernel** of the host operating system (OS).
- As a result, they avoid the infrastructure overhead of a full-blown OS and provide only those resources (i.e., **installations, dependencies,** and **code**) that your applications actually need.
- This means they can stop and start more quickly in response to **fluctuating scaling** requirements and offer better overall performance than VMs.

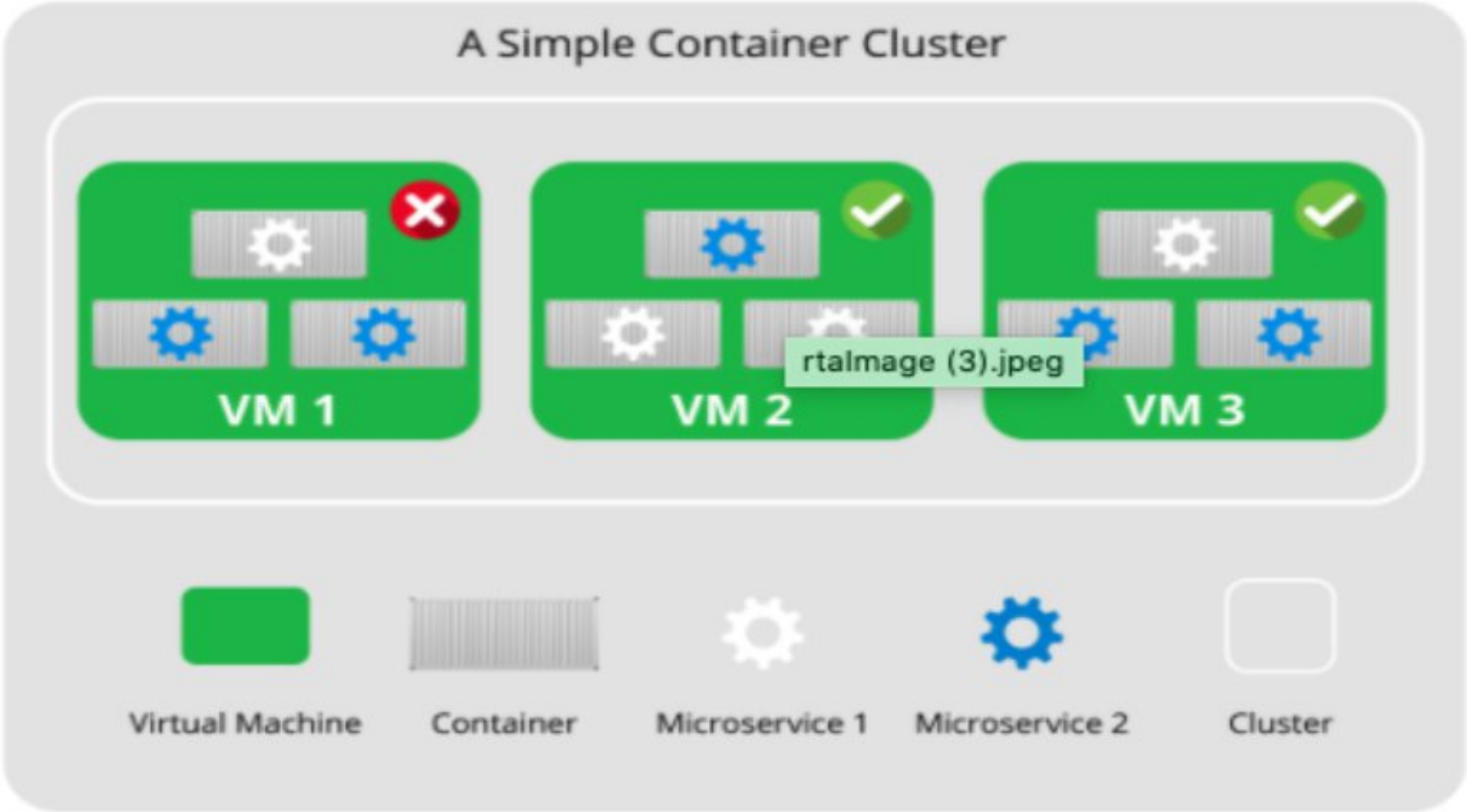
	Containers	Microservices
Communication	Use a single network for easier maintenance and configuration of multiple servers	Modular nature allows individual clusters or services to be on separate networks with their own address spaces
Scalability	Containers can scale easily to adapt to changing workloads, and they can be deployed, replicated, and managed easily. This allows applications to adapt to changing loads with minimal effort.	Microservices can scale independently, allowing each component of an app to be scaled depending on the resources it needs. This can efficiently use resources instead of having multiple copies of the entire application.
Management	Container management involves organizing, scaling, and maintaining containerized applications across various environments. This includes container orchestration, which automates the deployment, networking, scaling, and lifecycle management of containers.	Microservices performance testing is a critical aspect of developing and maintaining microservices-based applications. This involves assessing performance metrics such as response time, throughput, and the scalability of both individual microservices and the entire application.

Containers and microservices

- Containers and microservices are two important technologies driving the future of development.
- While they share many similarities, there are key differences that must be understood when determining which approach is right for an application.
- Containers enable portability and resource efficiency while offering great scalability options through containerization technology such as Docker or Kubernetes.
- On the other hand, Microservices allow flexibility due to their modular design, allowing applications to scale up without major changes in architecture all the while leveraging communication between services over a network connection; this also makes DevOps easier than ever before!

- To better performance and a lower infrastructure footprint, containerized microservices are typically more robust than a traditional monolithic application.
- For example, if you deploy an entire application to a VM, when the machine goes down, the whole application goes down with it. But, with containers, you can replicate microservices across a cluster of smaller VMs.

- So, in the event of a VM failure, the application will still be able to use the other microservices in the cluster and continue to function.
-


A Simple Container Cluster
VM 1
VM 2
VM 3
rtaImage (3).jpeg
Virtual Machine
Container
Microservice 1
Microservice 2
Cluster

Microservices → Many services → Many containers.

LOVELY PROFESSIONAL UNIVERSITY

Docker Compose helps:

- ✔ Define multiple services
- ✔ Manage networking
- ✔ Control dependencies
- ✔ Scale services
- ✔ Reproduce architecture easily

•Compose = Practical enabler of microservices (especially in dev/testing)

Docker Compose

Docker Compose is a tool used to define and manage multi-container

Docker applications using a YAML configuration file.

Key idea:

- ✔ **Instead of running containers individually**
- ✔ **Define the entire application stack declaratively**

Conceptual Understanding

Docker alone:

- **Runs individual containers**

Docker Compose:

- **Manages groups of related containers**

Its like:

- **Docker = Container-level management**

Docker Compose = Application-level management

Docker Compose follows:

- ✔ Declarative configuration
- ✔ Infrastructure as Code
- ✔ Reproducibility
- ✔ Automation

Need of Docker Compose

Problem Without Compose

Modern applications rarely use a single container.

Typical stack:

- ✔ **Web application**
  - ✔ **Database**
  - ✔ **Cache**
  - ✔ **Backend API**
  - ✔ **Message queue**

Running manually:

```
docker run web
   docker run db
   docker run redis
   docker run backend
```

Problems faces

Too many commands

- ✖ Hard to remember parameters
- ✖ Hard to reproduce setup
- ✖ Networking complexity
- ✖ Dependency issues

How docker solve the problem

(A) Multi-Container Management

Define entire system in one file.

- ✔ Centralized configuration
- ✔ Single command execution

(B) Reproducibility

Very strong academic point.

- ✔ Same environment across machines
- ✔ Same configuration for all developers
- "Works on my machine" problem → minimized.

(C) Simplified Networking

Without Compose:

✖ Manual network creation
✖ IP management

With Compose:

•✔ Automatic network creation
✔ Services communicate via names

Example:

•Service db reachable at hostname db.

•Huge conceptual advantage.

(D) Dependency Management

Control startup order.

- ✔ Web depends on DB
- ✔ Backend depends on Cache

Compose manages orchestration basics.

(E) Scalability

Scale services easily.

•docker compose up --scale web=3

•✔ No manual container duplication

(F) Cleaner Configuration

Instead of:

•docker run -d -p 8080:80 --name web ...

You define:

•ports:
  - "8080:80"

•✔ Human-readable

✔ Version-controlled

3. Basics of Docker Compose

Now move to fundamental building blocks.

(A) Compose File (docker-compose.yml)

Heart of Compose.

✔ Written in YAML
✔ Defines application stack

Basic structure:

```
version: "3.9"

services:
  service_name:
    configuration
```

•Important teaching note:

•YAML = indentation-sensitive
Spacing matters

(B) Services (Most Important Concept)

Service = Definition / Blueprint of containers

NOT containers directly.

•Example:

```
services:
  web:
    image: nginx
```

Conceptually:

•✔ Service defines role
✔ Container = Instance of service

Analogy:

•Class → Object
Service → Container

(C) image vs build

Two ways to specify container source.

Using Prebuilt Image

•image: nginx

•✔ Pull from registry

Building Custom Image

build: .

✔ Uses Dockerfile
✔ Builds locally

Theory distinction:

✔ image → Existing software
✔ build → Custom software

(D) ports

Defines host-container communication.

ports:
- "8080:80“

Explain carefully:

✓ HostPort : ContainerPort

Common student confusion → directionality.

(E) volumes

Persistent storage.

•volumes:
- data:/var/lib/mysql

•Concept:

•✔ Containers ephemeral
✔ Volumes persistent

•Benefits:

•✔ Data safety
✔ State preservation

(F) networks

Compose automatically creates:

- ✔ Default network

- **Conceptual advantages:**
- ✔ Service discovery
- ✔ Built-in DNS
- ✔ Name-based communication

- **Example:**
- **web → connect to db via hostname db**

(G) depends_on

Controls startup order.

•depends_on:
- db

•VERY IMPORTANT theory clarification:

•✓ Ensures start order
✖ Does NOT ensure readiness

How Docker Compose Works

Compose performs:

- **Reads YAML file**
- **Pulls/builds images**
- **Creates network(s)**
- **Creates volumes**
- **Starts containers**
- **Connects services**

Emphasize:

Compose automates container lifecycle management.

Docker Compose vs Docker CLI

Docker CLI	Docker Compose
Manual commands	Declarative file
Individual containers	Multi-container apps
Hard to reproduce	Easily reproducible
Verbose	Clean & structured

Basic Commands

Initialization & Setup

•docker-compose version
→ Check installed version

•docker-compose config
→ Validate YAML file syntax

Create the file : notepad docker-compose.yml

```
version: "3.9"
services:
  web:
    image: nginx
    ports:
      - "8080:80"
```

Starting Services

- **docker-compose up**
  → **Start all services (foreground)**
- **docker-compose up -d**
  → **Start in detached mode (background)**
- **docker-compose up --build**
  → **Rebuild images before starting**

Stopping Services

- docker-compose down
→ Stop & remove containers, networks
- docker-compose stop
→ Stop containers (no removal)
- docker-compose start
→ Restart stopped containers

Monitoring & Debugging

- docker-compose ps
→ List running services
- docker-compose logs
→ View logs
- docker-compose logs -f
→ Live logs

- **Executing Commands**
- **docker-compose exec <service> bash**
  → Enter container shell
- **docker-compose up --scale web=3**
  → Run multiple instances

A company wants to deploy a 3-tier web application using Docker Compose:

- Frontend → React (Node.js)
- Backend → Spring Boot API
- Database → PostgreSQL
- Requirements:

All services must communicate

- Environment variables should be used
- Data should persist using volumes
- Services must start in dependency order

- Create docker-compose.yaml file
- docker compose up –d
- docker compose config --services

- docker exec -it postgres-db psql -U postgres- check database

- **Create a Docker Compose setup for a Node.js application connected to MongoDB.**
  Requirements:
- **Node.js app runs on port 3000**
- **MongoDB runs on default port 27017**
- **Use environment variables for DB connection**
- **Ensure MongoDB starts before Node.js**

```
version: '3.8'
services:
  app:
    image: node:18
    container_name: node-app
    working_dir: /usr/src/app
    volumes:
      - .:/usr/src/app
    command: npm start
    ports:
      - "3000:3000"
    environment:
      - MONGO_URL=mongodb://mongo:27017/mydb
    depends_on:
      - mongo

  mongo:
    image: mongo:6
    container_name: mongo-db
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
```

- docker compose up -d
- docker compose ps
- docker compose logs app
- docker compose down

- **Deploy a WordPress website using Docker Compose with MySQL as the database.**
  Requirements:
- **WordPress runs on port 8080**
- **MySQL uses persistent storage**
- **Use environment variables for DB configuration**
- **Ensure proper service dependency**

```
version: '3.8'

services:
  wordpress:
    image: wordpress:latest
    container_name: wordpress-app
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wpuser
      WORDPRESS_DB_PASSWORD: wppass
      WORDPRESS_DB_NAME: wpdb
    depends_on:
      - db

  db:
    image: mysql:5.7
    container_name: mysql-db
    restart: always
    environment:
      MYSQL_DATABASE: wpdb
      MYSQL_USER: wpuser
      MYSQL_PASSWORD: wppass
```